

VidyaSetu — Intern Guide

VidyaSetu — Intern Guide

Welcome! This guide will help you set up the project, understand the codebase, and build the **Video Classroom** module.

About the Project

VidyaSetu is a Virtual Sanskrit Classroom platform for students and teachers.

- **Live demo:** https://nattsrk.github.io/SANSKRIT_VIRU
 - **Repo:** https://github.com/nattsrk/SANSKRIT_VIRU
 - **Stack:** React.js (frontend) + Node.js + Express (backend) + SQLite (database)
-

Step 1 — Setup Your Machine

Make sure you have these installed:

Tool	Version	Check
Node.js	v18+	<code>node --version</code>
npm	v9+	<code>npm --version</code>
Git	any	<code>git --version</code>
VS Code	latest	<code>code.visualstudio.com</code>

Clone and Run

```
# Clone the repo
git clone https://github.com/nattsrk/SANSKRIT_VIRU.git
cd SANSKRIT_VIRU
```

```
# Install all dependencies
npm run install-all
```

```
# Seed the database with sample data
npm run seed
```

Start both frontend and backend
npm run dev

Open your browser at **http://localhost:3000**

Demo Login Accounts

Role	Email	Password
Teacher	guru@vidyasetu.com	teacher123
Student	priya@vidyasetu.com	student123

Step 2 — Understand the Project Structure

```
SANSKRIT_VIRU/
├── client/                                     ← React frontend (runs on port 3000)
│   ├── src/
│   │   ├── context/
│   │   │   └── AuthContext.js               ← Login state + API calls
│   │   ├── components/
│   │   │   └── Navbar.js                   ← Top navigation bar
│   │   └── pages/
│   │       ├── Login.js                   ← Login & Register page
│   │       ├── Dashboard.js               ← Home after login
│   │       ├── Classes.js                 ← List of classes
│   │       ├── ClassDetail.js             ← Single class with lessons
│   │       └── Assignments.js             ← Assignments list
│   └── server/                             ← Node.js backend (runs on port 5001)
│       ├── server.js                       ← Entry point
│       ├── db/
│       │   ├── schema.js                  ← SQLite table definitions
│       │   └── seed.js                    ← Sample data
│       ├── middleware/
│       │   └── auth.js                    ← JWT token verification
│       └── routes/
│           ├── auth.js                    ← /api/auth/login, /register
│           ├── classes.js                 ← /api/classes
│           ├── assignments.js             ← /api/assignments
│           └── attendance.js              ← /api/attendance
└── docs/                                    ← Original project documents
```

Files to Read First (in this order)

1. `server/db/schema.js` — understand the database tables
2. `server/routes/auth.js` — how login/register works
3. `server/middleware/auth.js` — how JWT tokens protect routes

4. `client/src/context/AuthContext.js` — how frontend calls the API
 5. `client/src/pages/Dashboard.js` — a full page example
-

Step 3 — Learn These Concepts

Before coding, spend time understanding these topics.

React (2-3 days)

- Components, props, state (`useState`)
- Side effects (`useEffect`)
- React Router — `useNavigate`, `useParams`, `<Link>`
- Context API — how `AuthContext` shares login state globally

Practice: Add a simple “Hello [username]” message to the Dashboard.

Node.js + Express (1-2 days)

- How routes work: `router.get(...)`, `router.post(...)`
- Middleware: functions that run before route handlers
- How `req.body`, `req.params`, `res.json()` work

Practice: Add a `GET /api/health/details` endpoint that returns the current time and number of users in the DB.

Git Workflow (1 day)

Always work on a feature branch — never commit directly to main.

```
# Create your branch
git checkout -b feature/video-classroom

# After making changes
git add .
git commit -m "Add video room signaling"

# Push your branch
git push origin feature/video-classroom

# Then open a Pull Request on GitHub
```

Step 4 — Your Task: Video Classroom Module

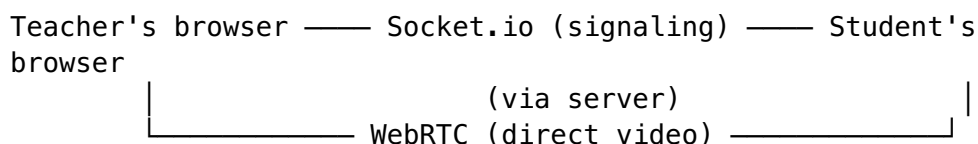
What You Are Building

A live video classroom where: - **Teacher** clicks “Start Class” → a video room opens
- **Students** see “Join Class” button → they join the teacher’s room - Everyone can see and hear each other (video + audio)

Technologies You Will Use

Technology	Purpose	Learn
WebRTC	Peer-to-peer video/audio between browsers	MDN WebRTC Guide
Socket.io	Real-time signaling (connect peers)	socket.io/docs

How WebRTC Works (Simple)



1. Teacher opens room → server is notified via Socket.io
2. Student joins → server tells teacher “someone wants to join”
3. Teacher and student exchange connection info (SDP + ICE candidates) via Socket.io
4. WebRTC connects them directly — video flows peer-to-peer

Tasks Breakdown

Task 1 — Learn WebRTC (2-3 days)

Build a simple standalone 2-person video call (separate from this project):

```
<!-- Just an HTML file, no framework needed -->
<!-- Use getUserMedia() to get camera -->
<!-- Use RTCPeerConnection to connect -->
```

Resources: - MDN: RTCPeerConnection documentation - Search: “simple WebRTC video call example”

Task 2 — Add Socket.io to the Server (1-2 days)

```
cd server
npm install socket.io
```

```
cd ../client
npm install socket.io-client
```

In server/server.js, add:

```
const { Server } = require('socket.io');
const http = require('http');

const httpServer = http.createServer(app);
const io = new Server(httpServer, { cors: { origin: '*' } });

io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  socket.on('join-room', (roomId) => {
    socket.join(roomId);
    socket.to(roomId).emit('user-joined', socket.id);
  });

  socket.on('offer', ({ to, offer }) => {
    io.to(to).emit('offer', { from: socket.id, offer });
  });

  socket.on('answer', ({ to, answer }) => {
    io.to(to).emit('answer', { from: socket.id, answer });
  });

  socket.on('ice-candidate', ({ to, candidate }) => {
    io.to(to).emit('ice-candidate', { from: socket.id,
      candidate });
  });

  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
  });
});

// Change app.listen to httpServer.listen
httpServer.listen(PORT, ...);
```

Test: open two browser tabs and verify user-joined fires.

Task 3 — Add DB Table + API (1-2 days)

Add to server/db/schema.js:

```
CREATE TABLE IF NOT EXISTS video_rooms (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  class_id INTEGER NOT NULL,
  host_id INTEGER NOT NULL,
  room_code TEXT UNIQUE NOT NULL,
  started_at DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
ended_at DATETIME,  
  FOREIGN KEY (class_id) REFERENCES classes(id),  
  FOREIGN KEY (host_id) REFERENCES users(id)  
);
```

Create `server/routes/rooms.js`: - POST `/api/rooms/start` — teacher starts a room - GET `/api/rooms/:classId/active` — check if a room is live - POST `/api/rooms/:id/end` — teacher ends the room

Task 4 — Build the React UI (3-4 days)

Create `client/src/pages/VideoRoom.js`: - Show your own video (local stream) - Show remote video(s) (other participants) - Mute / camera toggle buttons - Leave room button

Update `client/src/pages/ClassDetail.js`: - Teacher sees: **“Start Class”** button - Student sees: **“Join Class”** button (only when a room is active)

Add the route in `client/src/App.js`:

```
<Route path="/room/:roomId" element={<ProtectedRoute><VideoRoom /  
  ></ProtectedRoute>} />
```

Task 5 — Test + Polish (2 days)

- Test on two laptops connected to the same WiFi
 - Handle errors: camera denied, no microphone, connection lost
 - Show a “connecting...” state while WebRTC is setting up
 - Make sure “Leave Room” cleans up properly (stop tracks, close connection)
-

Step 5 — Submitting Your Work

1. Push your branch: `git push origin feature/video-classroom`
2. Go to https://github.com/nattsrk/SANSKRIT_VIRU
3. Click **“Compare & pull request”**
4. Write a short description of what you built and how to test it
5. Request a review

PR Checklist

☐

Video call works between teacher and student

☐

Teacher can start/end a room

☐

Student can join/leave

☐

- ☐ Camera and mic controls work
 - ☐ No console errors
 - ☐ Code is committed on feature/video-classroom branch
-

Useful Commands

```
npm run dev           # Start frontend + backend together
npm run seed          # Reset database with sample data
npm run server        # Start only backend
npm run client        # Start only frontend
```

Questions?

Raise a GitHub Issue on the repo or reach out to the project lead.

Good luck and happy coding!